Java Flow of Control - Comprehensive Deep Dive Notes

---

1. FLOW OF CONTROL

What is Flow of Control?

Flow of control refers to **the order in which individual statements, instructions, or function calls are executed** when a program is running.

When you write Java code, the JVM executes it through this pipeline:

```
Your Java Code (.java)
      ↓
  Java Compiler (javac)
      ↓
  Bytecode (.class file)
      ↓
  JVM reads bytecode
      ↓
Interpreter + JIT Compiler
      ↓
  Machine Code (executes)
```

During execution, the question is: **in what order does each line execute?**

Default Flow (Sequential)

By default, Java executes code **line by line, top to bottom**:

```
int i = 4;          // Step 1: i gets value 4
System.out.println(i); // Step 2: prints 4
int j = 5;          // Step 3: j gets value 5
System.out.println(j); // Step 4: prints 5
```

This is called **sequential execution** — the most basic form of flow of control.

---

2. TYPES OF FLOW OF CONTROL

Java has **three major categories** that alter or control the flow:

```
Flow of Control
├── 1. Selection (Conditional branching)
│       ├── if
│       ├── if-else
│       ├── if-else-if ladder
│       └── switch
│
├── 2. Iteration (Loops / Repetition)
│       ├── for loop
│       ├── while loop
│       ├── do-while loop
│       └── for-each loop
│
└── 3. Jumps (Skipping / Breaking flow)
        ├── break
        ├── continue
        └── return
```

---

3. SELECTION STATEMENTS (Conditionals)

The Core Idea

Imagine you're going home and the road **splits into two paths**:

- Path A: Long but scenic (trees, peaceful)
- Path B: Short but traffic-filled

Which path you take **depends on a condition** — how much time do you have?

- If time is plenty → Take Path A
- If time is less → Take Path B

This decision-making is exactly what selection statements do in code.

---

4. THE `if` STATEMENT

Syntax

```
if (expression) {
    // code to execute if expression is TRUE
}
```

How It Works

```
Program reaches the if statement
          ↓
    Evaluate the expression
          ↓
    Is it TRUE or FALSE?
      ↙         ↘
    TRUE         FALSE
      ↓           ↓
Execute the      Skip the block
 if block        entirely
      ↓           ↓
Continue with rest of code
```

Key Rules

- The expression inside `( )` **must evaluate to a boolean** — either `true` or `false`
- The curly braces `{ }` define the **block of code** that runs if condition is true
- If there's only **one statement** inside, `{ }` are optional (but always recommended)

Example

```
boolean b = true;

if (b == true) {
    System.out.println("b is true"); // executes only if b is true
}

System.out.println("This always executes"); // outside if, always runs
```

With Integer Comparison

```
int i = 5;
```

```
if (i == 5) {
    System.out.println("i is five"); // executes because i IS 5
}
```

Valid Expressions Inside `if`

```
// Equality check
if (i == 5) { }

// Greater than
if (i > 5) { }

// Less than
if (i < 5) { }

// Greater than or equal
if (i >= 5) { }

// Less than or equal
if (i <= 5) { }

// Logical AND (both conditions must be true)
if (i > 5 && i < 10) { }

// Logical OR (at least one must be true)
if (i == 5 || i == 10) { }

// NOT operator
if (!(i == 5)) { }
```

---

5. THE `if-else` STATEMENT

Syntax

```
if (expression) {
    // executes when expression is TRUE
} else {
    // executes when expression is FALSE
}
```

How It Works

```
Evaluate expression
      ↓
  TRUE or FALSE?
  ↙       ↘
 TRUE       FALSE
  ↓          ↓
Run if block  Run else block
  ↘       ↙
 Both paths merge
 Continue program
```

Important: Only ONE block ever executes. Never both.

Example — Odd or Even

```
int i = 9;

if (i % 2 == 0) {
    System.out.println("i is Even");
} else {
    System.out.println("i is Odd");
}
```

Why `i % 2`?

- The `%` is the **modulo operator** — it gives the **remainder** after division
- Even numbers: `i % 2 == 0` (no remainder when divided by 2)
- Odd numbers: `i % 2 == 1` (remainder of 1)

i = 6 → 6 % 2 = 0 → Even ✓
i = 9 → 9 % 2 = 1 → Odd ✓

---

6. CURLY BRACES `{ }` — Optional but Recommended

```
// This is valid (single statement, no braces needed):
if (i > 5)
    System.out.println("greater than 5");
else
    System.out.println("5 or less");
```

```
// This is also valid (mixed):
if (i > 5) {
    System.out.println("greater than 5");
} else
    System.out.println("5 or less");
```

Why Always Use Braces?

```
// DANGEROUS — no braces:
if (i > 5)
    System.out.println("Line 1"); // inside if
    System.out.println("Line 2"); // THIS IS OUTSIDE if — always runs!

// SAFE — with braces:
if (i > 5) {
    System.out.println("Line 1"); // inside if
    System.out.println("Line 2"); // inside if
}
```

Best Practice: Always use `{ }` even for single-line if blocks. It prevents bugs when you add more lines later.

---

7. NESTED `if` STATEMENTS

What is Nesting?

Placing an `if` **inside another** `if`. This allows checking multiple levels of conditions.

Syntax

```
if (condition1) {
    // outer if block
    if (condition2) {
        // inner if block — only reaches here if BOTH conditions are true
    } else {
        // inner else
    }
} else {
    // outer else
```

```
}
```

Example

```
int i = 8;

if (i > 5) {              // Check 1: is i greater than 5?
    if (i < 10) {         // Check 2: is i less than 10?
        if (i == 7) {     // Check 3: is i exactly 7?
            System.out.println("i is 7");
        }
        // No else here — if i != 7, nothing happens at this level
    } else {
        System.out.println("i is 10 or more");
    }
} else {
    System.out.println("i is 5 or less");
}
```

Flow Trace for `i = 8`

```
i = 8
  ↓
i > 5? → YES → enter outer if
  ↓
i < 10? → YES → enter inner if
  ↓
i == 7? → NO → skip innermost if (no else here)
  ↓
Exit inner if
  ↓
Exit outer if (don't go to outer else — outer condition WAS true)
  ↓
Program ends
```

Key insight: The `else` only executes if its corresponding `if` was **false**. Even if nothing happened inside the `if`, the `else` still won't run if the `if` condition was true.

Better Alternative — Use Logical Operators

Instead of nesting:

```
if (i > 5) {
    if (i < 10) {
        // ...
    }
}
```

Write it as:

```
if (i > 5 && i < 10) {
    // cleaner, same result
}
```

Nesting Rules

- You can nest to **any level** in Java
- But **good practice = maximum 2 levels** of nesting
- Deep nesting makes code hard to read and maintain

---

8. `if-else-if` LADDER

The Problem It Solves

`if-else` gives you **2 choices**. What if you need **5, 10, or 20 choices**?

Syntax

```
if (condition1) {
    // executes if condition1 is true
} else if (condition2) {
    // executes if condition1 false AND condition2 true
} else if (condition3) {
    // executes if condition1, 2 false AND condition3 true
} else if (condition4) {
    // executes if condition1, 2, 3 false AND condition4 true
} else {
    // executes if ALL above conditions are false (default case)
}
```

How It Works (Critical!)

```
Check condition1
   ↓ FALSE
Check condition2
   ↓ FALSE
Check condition3
   ↓ TRUE → Execute block 3 → EXIT the entire ladder
                              (skip all remaining else-ifs)
```

The moment one condition is TRUE, that block runs and the entire ladder is exited.

Example — Age Groups

```
int age = 50;

if (age > 80) {
   System.out.println("You are very old");
} else if (age > 60) {
   System.out.println("You are old");
} else if (age > 40) {
   System.out.println("You are becoming old");
} else if (age > 20) {
   System.out.println("You are young");
} else {
   System.out.println("You are a child");
}
```

Flow trace for `age = 50`:

```
age = 50
  ↓
age > 80? → NO
  ↓
age > 60? → NO
  ↓
age > 40? → YES (50 > 40) → print "You are becoming old" → EXIT ladder
```

Output: `You are becoming old`

Even though `age > 20` is also true (50 > 20), it NEVER gets checked because the ladder exits as soon as one condition matches.

Why Not Just Use Multiple Independent `if` Statements?

```
// WRONG APPROACH — independent ifs:
if (age > 80) { System.out.println("Very old"); }
if (age > 60) { System.out.println("Old"); }
if (age > 40) { System.out.println("Becoming old"); }  // age=50 → TRUE
if (age > 20) { System.out.println("Young"); }          // age=50 → ALSO TRUE!
```

Output (wrong):

```
You are becoming old
You are young
```

Both print because they're **independent conditions** — there's no ladder link between them.

The `else-if` ladder ensures only one block runs.

---

9. SWITCH STATEMENTS

What is Switch?

Switch is similar to an `if-else-if` ladder, but:

- More **optimized** internally
- Has a different syntax
- Works only with **specific data types**

Analogy

Like a **multi-way junction** in a road — you come from one direction and can go to path 1, 2, 3, or 4 based on your choice.

Syntax

```
switch (expression) {
   case value1:
      // code for case 1
      break;

   case value2:
      // code for case 2
      break;
```

```
    case value3:
        // code for case 3
        break;

    default:
        // code if no case matches (like the final 'else')
        break;
}
```

Component Breakdown

Component	Meaning
`switch(expression)`	The variable/value to compare
`case value:`	If expression == value, run this
`break`	Exit the switch block
`default`	Runs if no case matches (optional)

Example

```
int i = 3;

switch (i) {
    case 1:
        System.out.println("i is 1");
        break;
    case 2:
        System.out.println("i is 2");
        break;
    case 3:
        System.out.println("i is 3");
        break;
    default:
        System.out.println("i is greater than 3");
        break;
}
```

Flow for `i = 3`:

i = 3
↓
Compare with case 1: 3 == 1? NO → skip
↓
Compare with case 2: 3 == 2? NO → skip
↓
Compare with case 3: 3 == 3? YES → execute → break → exit switch

Output: `i is 3`

---

10. THE `break` KEYWORD — Critical!

What Happens Without `break`?

This is called **fall-through behavior**:

```
int i = 1;

switch (i) {
    case 1:
        System.out.println("i is 1");
        // NO BREAK HERE!
    case 2:
        System.out.println("i is 2");
        // NO BREAK HERE!
    case 3:
        System.out.println("i is 3");
        // NO BREAK HERE!
    default:
        System.out.println("i is greater than 3");
}
```

Output:

```
i is 1
i is 2
i is 3
i is greater than 3
```

All cases print! Even though only `case 1` matched.

Why Does Fall-Through Happen?

Without `break`, once a matching case is found, Java **keeps executing ALL subsequent cases** without checking their conditions. It just runs the code blocks one after another until it hits a `break` or the switch ends.

When is Fall-Through USEFUL?

```
int day = 3; // Wednesday

switch (day) {
    case 1:
    case 2:
    case 3:
    case 4:
    case 5:
        System.out.println("Weekday");
        break;
    case 6:
    case 7:
        System.out.println("Weekend");
        break;
}
```

Here, days 1-5 all fall through to print "Weekday". This is **intentional fall-through** — a very useful pattern!

With `break` (Normal Behavior)

```
int i = 1;

switch (i) {
    case 1:
        System.out.println("i is 1");
        break; // ← exits switch immediately
    case 2:
        System.out.println("i is 2"); // never reached
        break;
}
```

Output: `i is 1` (only)

11. DATA TYPES ALLOWED IN SWITCH

Before JDK 7

Switch expression could only be:

```
byte | short | int | char | enum

// VALID:
int i = 5;
switch (i) { }          // ✓ int

char c = 'A';
switch (c) { }          // ✓ char

byte b = 10;
switch (b) { }          // ✓ byte

// INVALID:
boolean flag = true;
switch (flag) { }       // ✗ ERROR: cannot switch on boolean

double d = 5.5;
switch (d) { }          // ✗ ERROR: cannot switch on double

long l = 100L;
switch (l) { }          // ✗ ERROR: cannot switch on long
```

After JDK 7

Strings are also allowed:

```
String day = "Monday";

switch (day) {
    case "Monday":
        System.out.println("Start of work week");
        break;
    case "Friday":
        System.out.println("End of work week");
        break;
```

```
    default:
        System.out.println("Middle of week");
}
```

After JDK 14

Switch was massively enhanced (covered separately):

- Arrow syntax (`case 1 ->`)
- Switch expressions (returns a value)
- No fall-through by default
- `yield` keyword

---

12. SWITCH RULES

No Duplicate Cases

```
switch (i) {
    case 1:
        // ...
    case 1:   // ✗ ERROR: Duplicate case
        // ...
}
```

Compare with `if-else-if`: duplicate conditions there are allowed (though logically wrong, no compile error):

```
if (i == 1) { }
else if (i == 1) { } // no error, just second block never executes
```

`default` is Optional

```
switch (i) {
    case 1:
        System.out.println("one");
        break;
    case 2:
        System.out.println("two");
        break;
    // no default — if i is 3, nothing happens
```

```
}
```

`default` Can Be Anywhere

```
switch (i) {
    default:        // can be first!
        System.out.println("other");
        break;
    case 1:
        System.out.println("one");
        break;
}
```

Though convention is to put `default` last.

---

13. SWITCH vs `if-else-if` LADDER — Deep Comparison

Surface Level Comparison

```
// if-else-if ladder:
if (i == 1) {
    // case 1
} else if (i == 2) {
    // case 2
} else if (i == 3) {
    // case 3
} else {
    // default
}

// Equivalent switch:
switch (i) {
    case 1: /* case 1 */ break;
    case 2: /* case 2 */ break;
    case 3: /* case 3 */ break;
    default: /* default */ break;
}
```

Both produce identical results. So why does switch exist?

Difference 1: What Can Be Tested

Feature	`if-else-if`	`switch`
Equality (`==`)	✓	✓
Greater than (`>`)	✓	✗
Less than (`<`)	✓	✗
Range checks (`> 5 && < 10`)	✓	✗
Boolean expressions	✓	✗

```
// ONLY possible with if-else-if:
if (age > 0 && age < 13) {
    System.out.println("Child");
} else if (age >= 13 && age < 20) {
    System.out.println("Teenager");
} else if (age >= 20) {
    System.out.println("Adult");
}
```

Switch can only do exact value matching.

Difference 2: Performance / Efficiency — The Deep Dive

This is where switch truly shines.

How `if-else-if` ladder executes internally:

Evaluate condition 1 → false
Evaluate condition 2 → false
Evaluate condition 3 → false
Evaluate condition 4 → TRUE → execute → exit

Total evaluations for 4th match: 4

Time complexity: **O(n)** — Linear. As cases grow, time grows proportionally.

How `switch` executes internally:

The Java compiler converts switch into something called a **Jump Table** internally.

---

14. JUMP TABLES — The Secret Behind Switch's Speed

What is a Jump Table?

Think of it like an **array where each index maps directly to a code block**:

Jump Table (array-like structure):
Index 0 → code block for case 1
Index 1 → code block for case 2
Index 2 → code block for case 3
Index 3 → default code block

When `i = 2`, instead of checking each case one by one:

- Calculate index: `i - 1 = 1`
- Directly jump to index 1 → execute case 2 immediately

Zero comparisons needed. Direct access.

Time complexity: **O(1)** — Constant time, regardless of number of cases!

Types of Jump Tables in Java

Java's compiler actually creates **two types** of jump tables:

Type 1: TableSwitch (Dense Values)

Used when case values are **close together** (dense):

```
switch (i) {
    case 1: /* ... */ break;
    case 2: /* ... */ break;
    case 3: /* ... */ break;
    case 4: /* ... */ break;
```

```
}
```

Values 1, 2, 3, 4 are consecutive → **Dense!**

Internal representation:

```
Index: [0]   [1]   [2]   [3]
Value:  1     2     3     4
Code: block1 block2 block3 block4
```

Direct formula: `jump to index (i - minValue)` → **O(1) lookup**

Type 2: LookupSwitch (Sparse Values)

Used when case values are **far apart** (sparse):

```
switch (i) {
   case 1:      /* ... */ break;
   case 1000:   /* ... */ break;
   case 100000: /* ... */ break;
}
```

Values 1, 1000, 100000 are far apart → **Sparse!**

Why not TableSwitch here? It would create a table with 100,000 slots where 99,997 are empty — massive memory waste!

LookupSwitch instead creates a compact sorted table:

```
Index: [0]     [1]      [2]
Value:  1      1000     100000
Code: block1  block2   block3
```

Then uses **Binary Search** to find the matching case:

- Go to middle: 1000. Is i < 1000? → search left half. Is i > 1000? → search right half
- Finds match in O(log n) time

Performance Summary

Method	Time Complexity	When Used
`if-else-if`	O(n)	Always
Switch (TableSwitch)	O(1)	Dense case values
Switch (LookupSwitch)	O(log n)	Sparse case values

Even O(log n) is better than O(n) for large numbers of cases.

How Compiler Decides

The compiler analyzes the switch cases and automatically chooses:

- If cases are dense → **TableSwitch**
- If cases are sparse → **LookupSwitch**
- You don't control this — it's automatic

---

15. NESTED SWITCH

What is it?

A switch statement **inside another switch statement**.

```
int j = 5;
int k = 1;

switch (j) {
    case 4:
        System.out.println("j is 4");
        break;

    case 5:
        // Nested switch inside case 5
        switch (k) {
            case 1:
                System.out.println("j is 5, k is 1");
                break;
            case 2:
                System.out.println("j is 5, k is 2");
```

```
            break;
        }
        break;

    default:
        System.out.println("j is not 4 or 5");
        break;
}
```

Flow for `j = 5, k = 1`:

```
switch(j) → case 5 matches
    ↓
Enter case 5 block
    ↓
switch(k) → case 1 matches
    ↓
Print "j is 5, k is 1"
    ↓
break (exits inner switch)
    ↓
break (exits outer switch)
```

Triple Nesting (Possible but Discouraged)

```
switch (a) {
    case 1:
        switch (b) {
            case 1:
                switch (c) {  // 3 levels deep
                    case 1: /* ... */ break;
                }
                break;
        }
        break;
}
```

When to Use Nested Switch

Only when your logic genuinely requires it — for example:

- First switch selects a **category**

- Inner switch selects a **subcategory**

Best Practice Warning

Just like nested `if`, avoid deep nesting in switch too. Maximum 2 levels recommended for readability.

---

16. COMPLETE COMPARISON SUMMARY

if vs if-else vs if-else-if vs switch

Feature	`if`	`if-else`	`if-else-if`	`switch`
Choices	1 (or skip)	Exactly 2	Multiple	Multiple
Condition type	Any boolean	Any boolean	Any boolean	Equality only
Data types	Any	Any	Any	byte, short, int, char, String, enum
Performance	O(n)	O(1)	O(n)	O(1) or O(log n)
Fall-through	No	No	No	Yes (needs break)
Duplicate conditions	Allowed	Allowed	Allowed	NOT allowed
Range checks	Yes	Yes	Yes	No
Null safety	Yes	Yes	Yes	NullPointerException risk with String

---

17. PRACTICAL DECISION GUIDE

When to Use `if`

```
// Single condition, simple check
if (isLoggedIn) {
    showDashboard();
}
```

When to Use `if-else`

```
// Binary choice
if (age >= 18) {
    System.out.println("Adult");
} else {
    System.out.println("Minor");
}
```

When to Use `if-else-if`

```
// Multiple conditions with ranges or complex boolean logic
if (score >= 90) {
    grade = "A";
} else if (score >= 80) {
    grade = "B";
} else if (score >= 70) {
    grade = "C";
} else {
    grade = "F";
}
```

(Can't do this with switch — ranges aren't supported!)

When to Use `switch`

```
// Multiple exact value checks on same variable — faster!
switch (dayOfWeek) {
    case 1: System.out.println("Monday"); break;
    case 2: System.out.println("Tuesday"); break;
    case 3: System.out.println("Wednesday"); break;
    // etc.
}
```

---

18. COMMON MISTAKES & HOW TO AVOID THEM

Mistake 1: Forgetting `break` in switch

```
// BUG:
switch (i) {
    case 1:
```

```
        System.out.println("one");
        // forgot break! falls through to case 2
    case 2:
        System.out.println("two"); // prints even when i=1
}
```

Mistake 2: Using `=` instead of `==`

```
// BUG (assignment, not comparison):
if (i = 5) { }  // compile error in Java (unlike C++)

// CORRECT:
if (i == 5) { }
```

Mistake 3: Semicolon after `if`

```
// BUG:
if (i == 5);            // semicolon ends the if — empty body!
    System.out.println("always prints");  // always executes

// CORRECT:
if (i == 5)
    System.out.println("prints only when i is 5");
```

Mistake 4: Deeply nested if/switch

```
// Hard to read (bad practice):
if (a) {
    if (b) {
        if (c) {
            if (d) {
                // ...
            }
        }
    }
}

// Better: Use logical operators
if (a && b && c && d) {
    // ...
}
```

---

19. QUICK REFERENCE — Syntax at a Glance

```
// ═══════════════════════════════════════════
// SIMPLE IF
// ═══════════════════════════════════════════
if (condition) {
    // runs if true
}

// ═══════════════════════════════════════════
// IF-ELSE
// ═══════════════════════════════════════════
if (condition) {
    // runs if true
} else {
    // runs if false
}

// ═══════════════════════════════════════════
// IF-ELSE-IF LADDER
// ═══════════════════════════════════════════
if (condition1) {
    // ...
} else if (condition2) {
    // ...
} else if (condition3) {
    // ...
} else {
    // default
}

// ═══════════════════════════════════════════
// SWITCH
// ═══════════════════════════════════════════
switch (variable) {
    case val1:
        // code
        break;
    case val2:
        // code
        break;
    default:
        // code
        break;
```

```
}

// ═══════════════════════════════════════════════
// NESTED IF
// ═══════════════════════════════════════════════
if (outer) {
    if (inner) {
        // both must be true
    }
}

// ═══════════════════════════════════════════════
// NESTED SWITCH
// ═══════════════════════════════════════════════
switch (a) {
    case 1:
        switch (b) {
            case 1: /* ... */ break;
        }
        break;
}
```

---

20. KEY TAKEAWAYS

1. **Flow of control** determines the order code executes — normally sequential (line by line)

2. **Selection statements** create branching — the code can go different directions based on conditions

3. `if` gives one optional branch; `if-else` gives exactly two branches; `if-else-if` **ladder** gives multiple branches

4. **Switch** is equivalent to `if-else-if` ladder for equality checks but is **faster** due to jump tables

5. `break` in switch prevents fall-through; intentional fall-through is useful for grouping cases

6. **Jump tables** (TableSwitch = O(1), LookupSwitch = O(log n)) are why switch outperforms the ladder's O(n)

7. **Always use curly braces** even for single-line blocks — prevents future bugs
8. **Avoid deep nesting** (max 2 levels) — use logical operators instead
9. After JDK 14, enhanced switch brings even more power (covered separately)